

FLOQR Technical Bible

Field	Value
Document	FLOQR Technical Bible
Version	1.0.0
Date	2026-07-22
Classification	Internal — Engineering / Architecture / Security
Audience	Engineering, Product, Security, DevOps, Investors (technical DD), Future CTO
Status	Canonical blueprint (living document)
Grounded codebase	work/v29-08 (package lineage v29.x)
Live demo	https://jadzadco.github.io/shoutout-demo/
Firebase project	shoutoutdemo-5b402
Functions region	us-central1 (Node.js 22)

How to read this document

- **Current (v29.x)** = what ships today (GitHub Pages static app + Firebase Auth/Firestore/Storage/Functions).
 - **Target** = production architecture for millions of patrons, tens of thousands of venues, multi-region.
 - When multiple options exist: **Recommendation** first, then **Alternatives**, then **Migration path**.
 - This is an engineering reference, not a pitch deck.
-

Table of Contents

1. [System Philosophy](#)
2. [Complete Product Architecture](#)
3. [User Roles](#)
4. [Complete Firestore Design](#)
5. [Backend](#)
6. [Mobile Architecture](#)
7. [Web Architecture](#)
8. [AI Architecture](#)
9. [Hardware](#)
10. [APIs & Integrations](#)

11. [Security](#)
 12. [Observability](#)
 13. [DevOps](#)
 14. [Scalability](#)
 15. [Technical Roadmap](#)
 16. [Engineering Recommendations](#)
-

SECTION 1 — System Philosophy

1.1 Microservices vs Modular Monolith

Recommendation

Start and grow as a Modular Monolith with hard domain boundaries, then extract microservices only when a boundary has independent scale, failure, or release cadence needs.

For FLOQR today, that means:

- One **API/runtime plane** initially (Cloud Functions + Cloud Run services co-located), organized by domain packages: identity, venues, shoutout, mingl, bartr, rydr, floqmedia, billing, ai, devices.
- One **web client monorepo** (or Flutter multi-package workspace) with shared design system and domain SDKs.
- Extract first candidates later: **Display/Device Gateway, Media Transcode, Realtime Chat, Payments Webhooks, AI Inference, DOOH Ad Decisioning**.

Why

- FLOQR domains share identity, venues, events, notifications, and billing. Premature microservices multiply auth, schema, and ops cost before product-market fit at scale.
- Modular monolith preserves transactional consistency for critical flows (checkout → order → ledger; ShoutOut submit → queue → approve → liveContent).
- Extraction remains possible if packages never import across domain internals (only public ports).

Alternatives

Approach	When it wins	Cost
Pure microservices day one	Large platform team already staffed	High ops, distributed transactions
Classic big-ball monolith	Tiny prototype only	Becomes unmaintainable past ~10 engineers
Serverless-only glue	Spike velocity	Harder local reasoning, cold starts, sprawl

Migration path

1. Enforce package boundaries + CI import lint (no cross-domain deep imports).
 2. Publish domain events to Pub/Sub even inside monolith.
 3. Split hottest consumer into Cloud Run + private VPC when p95 or deploy risk demands it.
 4. Keep Firestore as system of record until a domain needs its own store (chat fanout, time-series telemetry).
-

1.2 Domain-Driven Design (DDD)

Bounded contexts (target)

Context	Core language	Owns
Identity & Access	Patron, Handle, Role, Session, MFA	AuthN/AuthZ, profiles, consents
Venue Ops	Club, Location, Screen, Admin Assignment	Venues, staff, REP, guest lists
ShoutOut	Template, Board, Queue, Live Content	Messaging to displays
Social Graph	Mingl, Connection, Gist, Chat	Relationships & presence
Commerce (BartR)	Listing, Order, Fulfillment, Settlement	Marketplace
Mobility (RydR)	Ride, Pickup Zone, Dispatch	Transport
FloqMedia	Campaign, Creative, Placement, Impression	DOOH ads
Billing	Checkout, Ledger, Payout	Money movement
AI Platform	Signal, Recommendation, Agent Job	Models & features
Device Fleet	Device, Firmware, Telemetry	Hardware

Current mapping (v29.x)

- Identity: users, email OTP, MFA enrollment, master email allowlists.
- Venue Ops: clubLocations, clubAdminAssignments, clubEmployeeDesignations, admin.html.
- ShoutOut: shoutouts, liveContent, templates, display.html.
- Social: minglConnections, minglGists, chatRooms, chatMessages.
- Commerce: commerceProducts, serviceOrders, paymentLedger.
- Mobility: pickupRequests + Tesla robotaxi simulation.
- FloqMedia: spotAdCampaigns + club marketing campaigns.
- AI: aiIndex, crawl runs, recommend callables.

Anti-corruption layers

External systems (Stripe, Twilio, Xibo, Ticketmaster, Meta) must never leak into domain documents as primary keys without an opaque binding table (pattern already started with stripeConnectAccounts).

1.3 Service Boundaries

Hard rules

1. **Write authority:** only the owning service writes its collections (enforced by Functions + Security Rules).
2. **Read models:** other domains consume via APIs or projected read collections, never ad-hoc client joins across 8 collections in production clients.

3. **Sync communication** for user-facing request/response; **async events** for fan-out (notify club, expire live content, index search, settle payouts).

Recommended boundary checklist before extraction

- Separate SLO (e.g., display playback ≠ marketplace browse).
- Separate scaling axis (CPU-heavy media vs chat fanout).
- Separate compliance boundary (PCI-adjacent vs social graph).
- Team ownership exists.

1.4 Shared Services

Shared platform capabilities (never product-specific):

Shared service	Responsibility
Identity Service	Auth, sessions, MFA, handles, roles
Config / Feature Flags	Global + entity gates (already: platformSettings, venue/patron gates)
Notification Fabric	In-app, push, email, SMS, WhatsApp
Media Service	Upload, virus scan, transcode, CDN URLs
Search Service	Unified index over venues, people, products, events
Analytics Pipeline	Event ingest → BigQuery
Billing Kernel	Checkout session factory, webhooks, ledger
Audit Log	Tamper-evident security and admin actions
Device Registry	Pairing, heartbeats, OTA
AI Gateway	Prompt routing, quotas, safety filters

1.5 Event-Driven Architecture

Recommendation

Domain events as first-class contracts on Google Cloud Pub/Sub (or Eventarc), with Cloud Tasks for delayed work and Scheduler for crons.

Current (v29.x)

- Mostly **synchronous Firestore listeners + callables**.
- Examples of event-like behavior: onShoutoutCreatedNotifyClub, expireLiveShoutouts, scheduled AI crawl, daily auth code rotation.

Target event catalog (selected)

Event	Producers	Consumers
shoutout.submitted	ShoutOut API	Notify club, analytics, AI summarize
shoutout.approved	Venue Admin	liveContent writer, display devices, ledger
shoutout.expired	Scheduler	Reset idle board
order.paid	Stripe webhook	Inventory, fulfillment, seller notify
order.shipped	Fulfillment	Buyer notify, analytics
mingl.connection.accepted	Social	Chat room provision
venue.admin.assigned	Master Admin	MFA enroll reminder
device.heartbeat.missed	Device fleet	Ops alert
campaign.impression	Display/Ad SDK	Billing, analytics

Alternatives

- Firestore triggers only — simple, but hard to replay and fan out multi-region.
- Kafka self-managed — overkill until high throughput ad/telemetry.

Migration

Emit events from Functions beside existing writes (dual-write), add consumers gradually, then remove direct coupling.

1.6 CQRS

Recommendation

Selective CQRS, not global.

Use command models for writes (strict validation) and **read models** for:

- Patron discovery feeds
- Venue dashboards
- Ad decisioning
- Search
- Reporting

Current

Clients often query Firestore collections directly (command $\approx$ query). Acceptable for demo; will not hold at millions of users with complex joins.

Target

- Commands: Cloud Run HTTP/gRPC + App Check.
 - Queries: denormalized Firestore docs, Memorystore, or BigQuery materialized for analytics UIs.
 - Projection workers update read models from domain events.
-

1.7 Pub/Sub

Recommendation

Google Cloud **Pub/Sub** as the backbone for domain events + telemetry.

Topics grouped by domain (`floqr.shoutout.*`, `floqr.commerce.*`). Use **ordering keys** for per-device or per-order sequences. Dead-letter topics mandatory.

1.8 API Gateway

Recommendation

Apigee or Cloud Endpoints / API Gateway in front of Cloud Run for public APIs; Firebase Auth tokens validated at edge.

Layers:

1. Edge CDN (Cloud CDN / Cloudflare) for static + media.
2. API Gateway: auth, rate limit, WAF, request logging.
3. Service mesh optional later (ASM) once multi-service.

Current

Direct browser → Firebase SDKs + HTTPS Functions URLs. Fine for demo; weak for partner API productization.

1.9 Edge Computing

Recommendation

- Static UI + template assets on CDN.
 - Geo-routed API regions (see Global Deployment).
 - **Display players** as edge nodes: cache last approved creative, play offline, sync when online.
 - Future: Cloudflare Workers / Cloud Run jobs near venue metros for ad decisioning <50ms.
-

1.10 Offline Support

Critical offline surfaces

1. **LED display players** (must never go black on WAN blip).
2. **Door / guest-list check-in** tablets.
3. **Mobile patrons** in poor venue Wi-Fi (graceful degrade, not full offline social graph).

Recommendation

- Displays: local SQLite/IndexedDB queue of last N approved payloads + idle brand loop.
 - Mobile: Flutter offline cache for profile, tickets, recent chats metadata; full chat sync when online.
 - Conflict policy: **server wins** for money and approvals; **LWW with vector clocks** only for non-critical prefs.
-

1.11 Synchronization

Data class	Sync mode
liveContent to displays	Realtime snapshot + heartbeat reconcile
Orders / payments	Strict online; webhook is source of truth
Chat	Realtime + catch-up pagination
Catalog browse	Cache-first CDN/API with ETag
Device firmware	Staged OTA with ack

1.12 Scalability Philosophy

1. **Scale reads with denormalization and CDN**, not heroic client queries.
 2. **Scale writes by partitioning** (venueId, region, shard).
 3. **Isolate hot paths** (display ingest, checkout webhook, chat fanout).
 4. Prefer **horizontal** Cloud Run + Pub/Sub over bigger single Functions instances.
 5. Cost-aware: BigQuery for analytics, not Firestore scans.
-

1.13 High Availability

Target SLOs (initial production)

Surface	Availability	Notes
Display playback local	99.95%	Offline cache counts

Checkout	99.9%	Stripe dependency
Patron browse	99.9%	Multi-region
Admin portals	99.5%	

Multi-region active-active for Auth + CDN; Firestore multi-region; Functions/Run regional with failover runbook.

1.14 Fault Tolerance

- Idempotent webhooks (`stripeWebhookEvents` already).
 - Retry with jitter on Tasks.
 - Circuit breakers on partner APIs.
 - Poison message quarantine.
 - Display fallback creative on error (brand slide / idle template — Heist pattern already exists).
-

1.15 Observability

See Section 12. Philosophy: **every money, approval, and device state change is traceable** with correlation IDs (`requestId`, `orderId`, `shoutoutId`, `deviceId`).

1.16 Global Deployment Strategy

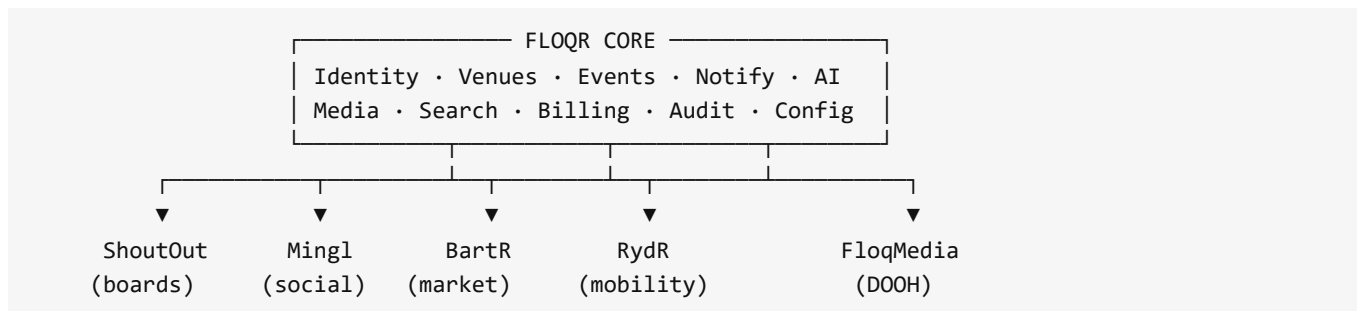
Recommendation

Tier	Regions
Phase 1	us-central1 + us-east1
Phase 2	Add europe-west1, asia-southeast1
Data residency	Venue org chooses region; PII stays in-region where required

Traffic: Geo DNS / Anycast CDN → nearest gateway → regional services. Shared global directory of venue IDs with region pointers.

SECTION 2 — Complete Product Architecture

2.0 Product topology



Independently deployable **at the service boundary level**; shared **data contracts** via events and core APIs.

2.1 FLOQR Core

Authentication

Current: Firebase Auth (Google, Microsoft, Facebook, phone SMS, email OTP → custom token). Club Admin MFA. Master SOS2FA.

Target: Same IdP core + optional enterprise SAML/OIDC for venue chains; WebAuthn for high-privilege admins; session risk scoring.

Recommendation: Keep Firebase Auth as consumer IdP; mint **FLOQR session JWT** (short-lived) via Identity Service for non-Firebase clients (Flutter, partners). Migrate custom claims for `masterAdmin`, `roles[]`, `venueScopes[]`.

Profiles / Identity

- Canonical `users/{uid}` with FloqR/Mingl handle uniqueness index.
- Public projection `publicProfiles/{handle}` for discovery (PII stripped).
- Identity graph: `uid ↔ handles ↔ phone hashes ↔ Stripe customer id` (binding table).

Permissions / Organizations / Venues

- Org hierarchy: `Organization → Brand → Venue (clubLocation) → Screens`.
- Assignments: `clubAdminAssignments` (elected) is correct pattern — keep; expand to org-level RBAC.

Events

Unified events collection + Ticketmaster/Eventbrite imports as sources of truth projections.

Notifications / Messaging

Notification Fabric channels: in-app (`inboxNotifications`), push (FCM), email (SendGrid), SMS/WhatsApp (Twilio) with credit wallets (`clubMessagingCredits`).

Media

Upload → Storage → async virus scan + transcode → CDN. Gemini edit jobs recorded in `aiMediaEdits`.

Search

Target: Vertex AI Search / Elasticsearch/OpenSearch over venues, people, products, events. Current: client-side + AI crawl index (`aiIndex`).

Analytics / Audit / Reporting

- Analytics: event stream → BigQuery.
- Audit: append-only `auditLogs` (extend beyond `appLogs` / `shoutoutAudit`).
- Reports: scheduled queries + exported CSV for venue finance.

AI Services

AI Gateway (Section 8) with per-tenant quotas.

Billing / Configuration / Administration

- Billing kernel owns Checkout + ledger.
- Feature gates already exist — graduate to LaunchDarkly-style or Firebase Remote Config + server enforcement.
- Master Admin = break-glass + entity management; never embed secret ops in client alone.

2.2 ShoutOut

End-to-end workflow (target)

1. Patron selects venue + screen format + template.
2. Compose text/media within layout contracts (`FLOORTextLayout`).
3. Price resolve → Checkout (if paid) or free submit.
4. shoutouts doc pending → club notify (SMS/WhatsApp/in-app).
5. Club Admin approve/reject (daily auth codes for SMS ops).
6. On approve: write `liveContent/{locationId}` with duration; display listeners update.
7. Expire job restores idle/brand loop.

8. Ledger accrual for paid.

Media processing

Upload, optional Gemini enhance, trim metadata for video, football intro pipeline for Zebbies advanced template.

Template Engine

- Shared templates + venue-restricted sets (`restrictTemplatesToLocationSet`).
- Text profiles per screen format (critical for LED readability).
- Club background variants when Master marks editable.
- **Target:** versioned template packages (JSON + assets + shaders) with marketplace SKUs.

Scheduling / Moderation / Approval / Queue

- Queue = pending shoutouts query by location.
- Moderation: keyword lists + AI classifier pre-check + human approve.
- Scheduling: future `scheduledAt` for timed takeovers (New Year, halftime).

Publishing pipeline / Display communication

- Primary: Firestore `liveContent` realtime.
- **Device Gateway** (target): HTTPS long-poll / MQTT / WebSocket to players behind NAT.
- Payload signed; devices verify.

Amazon Signage / Xibo

Current: Xibo Webpage widget loads `display.html?location=...` (documented in `XIBO-ZEBBIES-FOOTBALL-SETUP.md`).

Recommendation: Keep HTML player as universal renderer; add native players later. Amazon Signage: treat as another Webpage/HTML5 host or SMPTE-compatible package export.

Offline publishing / Playback sync

Cache last approved creative; clock-sync via NTP; optional SMPTE timecode for multi-panel arenas later.

Digital Signage management

Device registry (`displayDevices` exists), bind IP/location, remote screenshot (future), reboot command, health.

Template marketplace / Sponsor templates

Paid/sponsored templates with brand safety and revenue share to template authors.

Animation engine / Media optimization / Battery-aware

- Prefer CSS/Canvas/WebGL constrained profiles per panel size.
- Transcode to H.264 baseline for Xibo MP4 path.
- Battery: reduce FPS, dim, defer downloads on low SOC (device telemetry).

Device registration / Remote diagnostics / Telemetry

Pairing code → claim device → heartbeat metrics (temp, SOC, network, last frame hash).

2.3 Mingl

Matching / Discovery / Recommendations

Signals: interests, music, venues visited, follows, Gist text. Rank with AI + rules. Privacy: only opted-in fields.

Relationship graph

friendRequests / friendships / minglConnections — consolidate to one graph store at scale (or Neo4j/Spanner graph later). For now Firestore edges with composite indexes.

Venue presence

Optional check-in / geofence presence for discovery boosts (explicit consent).

Chat / Groups / Communities

chatRooms / chatMessages — extract to dedicated realtime service before 100k concurrent chats.

Moderation / AI suggestions

Report flows, rate limits, AI toxicity scoring, CSR tools.

2.4 BartR

Marketplace / Catalog

Public browse (**target UX**): guests browse catalog; auth only at purchase. Products in commerceProducts already allow read: if true.

Orders / Inventory / Payments / Settlement

- Checkout Session MoR (floqr-platform).

- Inventory decrement on paid webhook (idempotent).
- Settlement accrued to vendors; batch payouts via Connect.

Escrow (future)

Hold funds until delivery confirmation for high-value physical goods; state machine authorized → captured → released.

Ratings / Digital & Physical goods

Digital: Mixcloud/playlist licenses (policy in DJ-BARTR-MIXCLOUD.md). Physical: requiresShipping → Stripe shipping collection → shippingStatus.

Vendor onboarding / Fraud / Disputes / Shipping

Connect onboarding; velocity checks; dispute workflow tied to Stripe chargebacks; Shippo/EasyPost/UPS APIs for labels (target).

2.5 RydR

Current

Robotaxi pickup **simulation** (requestTeslaRobotaxiPickup, pickup.html) — no Stripe.

Target architecture

- Ride request with venue event context (end-of-night surge).
 - Geofenced pickup zones per venue.
 - ETA from Maps + traffic.
 - Dispatch adapter interface: Tesla / Waymo / Uber APIs.
 - Driver mode later for human fleets.
 - Event-aware batching (shared rides to metro).
-

2.6 FloqMedia

Current

Spot ad campaigns (spotAdCampaigns), club marketing SMS/WhatsApp, interstitial slots in app.

Target DOOH stack

Campaign Manager → Creative approval → Audience/venue targeting → Placement on SponsorWall / WindowAds / StreetCast / Arena → Impression beacons → Revenue share → Programmatic exchange

adapters → AI bid/optimize.

Decisioning service must be edge-capable (<50ms) with fallback house ads.

SECTION 3 — User Roles

Legend: **AuthN** = how they sign in; **AuthZ** = policy source.

For each role: Responsibilities · Permissions · Capabilities · Auth · Dashboard · Notifications · Reports · Security · Collections · APIs.

Guest

- Browse public club profiles, BartR catalog, limited discovery.
- No PII write; no checkout until account.
- Auth: none / anonymous App Check.
- Security: rate limit IP; bot detection.
- Collections (read): public projections, `commerceProducts` active.
- APIs: public catalog, public venue profile.

Patron

- Core consumer: ShoutOut, Mingl, BartR buy, RydR request, tickets.
- Auth: Firebase social/phone/email OTP.
- Permissions: self profile; own orders; own chats.
- Dashboard: patron portal.
- Collections: `users` self, `shoutouts` own, `serviceOrders` own, `mingl` edges.
- Security: verified email for money; optional MFA.

VIP Patron

- Same as Patron + entitlements (skip lines, VIP templates, priority support).
- AuthZ: `patronRanks` / subscription entitlement claims.
- Reports: none admin; personal spend summary.

Venue Owner

- Legal/org owner of venue(s); may delegate admins.
- Permissions: assign Venue Administrators; billing for club Stripe; contracts.
- Dashboard: org console (target); today often same as Master-assisted onboarding.
- Security: strong MFA; org verification KYC.

Venue Administrator (Club Admin)

- Elected via Master (`assignClubAdmin` → `clubAdminAssignments`).
- Responsibilities: approve ShoutOuts, public profile, guest lists, staff REP, ads credits, reconciliation view.

- Auth: Firebase + **required SMS MFA** (non-master).
- Dashboard: `admin.html` Venue Command Center (location-scoped).
- Notifications: pending shoutouts via preferred channels.
- Reports: audience, promoter, reconciliation ledger read.
- Collections: location-scoped shoutouts, `liveContent`, `guestList*`, `clubMedia`, `paymentLedger` read for club.
- APIs: admin callables; messaging credits; marketing send.
- **Hard rule:** no cross-venue entity search (Master only).

Regional Administrator

- Target role: manages venues in a geography for a chain/org.
- Permissions: subset of Master within `regionId`.
- Auth: SSO + MFA.
- Dashboard: regional ops console.

Master Administrator

- Platform superuser; entity onboarding; feature gates; diagnostics; SOS2FA for sensitive panels.
- Auth: allowlisted emails + domain + provider policy + SOS2FA.
- Collections: broad read; gated writes.
- APIs: `assignClubAdmin`, feature gates, crawler, purge test payments.

CSR

- Club-designated support inbox for venue.
- Permissions: read/respond club support threads; no finance.
- Dashboard: CSR queue in admin/portal.
- Security: least privilege; transcript retention policy.

Moderator

- Platform or venue content moderation (Mingl, ShoutOut text, UGC).
- Permissions: hide/remove content; suspend users within scope.
- Audit all actions.

Bottle Girl / Cocktail Waitress / Bartender / Host / Security

- Service staff; elect into club; Club Admin approves.
- Permissions: REP-scoped (guest list assist, public post with approval).
- **Do not** receive full Club Admin portal (v29.09.43+ access gate).
- Collections: `workerAssociationRequests`, `clubEmployeeDesignations`, `clubRoleActivity`.

DJ

- Profiles, lineups, BartR digital goods, featured on club pages.
- Collections: djProfiles, commerce products, club media relations.

Promoter

- Guest-list campaigns, referral attribution, promoter admin dashboard.
- Collections: promoterProfiles, guestList*, referral metrics.
- Dashboard: promoter-admin.html.

Artist / Photographer / Content Creator

- Media rights, featured people, BartR licensing SKUs.
- Permissions: upload owned media; license metadata.

Advertiser

- FloqMedia campaigns; spend; creatives; analytics.
- Auth: advertiser org accounts; billing profile.
- APIs: campaign CRUD, reporting.

Vendor (BartR seller)

- US marketplace eligibility; Connect onboarding; catalog; fulfillment.
- Collections: commerceProducts, seller views of serviceOrders.
- Security: Connect KYC; payout holds.

Driver / Fleet Manager

- RydR human fleet (future); accept trips; fleet telemetry.
- Auth: driver app + background check status flags.

Developer

- API keys, webhooks, sandbox.
- Auth: developer portal; scoped OAuth clients.
- Security: secret rotation; least scope.

Support / Operations / Executive

- Support: Zendesk-like tools + Master read.
 - Operations: device fleet, incidents, SLOs.
 - Executive: BI dashboards (BigQuery), no raw PII exports without control.
-

SECTION 4 — Complete Firestore Design

4.1 Naming conventions

Pattern	Rule
Collections	lowerCamelCase plural nouns (clubLocations)
IDs	Stable kebab for venues (heist-washington-dc); uid for users; {venueId}_{uid} for assignments
Fields	lowerCamelCase; timestamps createdAt/updatedAt serverTimestamp
Money	integer *Cents
Booleans	positive names (active, commerceEnabled)
Soft delete	status: "offboarded" "disabled" + deletedAt — avoid hard delete for money/audit
PII	never in public projections

4.2 Versioning

- Document schema schemaVersion: 1.
- Template packages templateVersion.
- Rules file header version markers (already practiced).
- API Accept-Version for public HTTP.

4.3 Current collections catalog (v29.x)

users, clubs, clubLocations, platformSettings, clubLocationAliases, clubAdminAssignments, emailOtpChallenges, events, templates, shoutouts, liveContent, displayDevices, guestListRequests, guestListCampaigns, commerceProducts, serviceOrders, appLogs, paymentLedger, stripeConnectAccounts, stripeWebhookEvents, entityFollows, audienceCampaigns, pickupRequests, clubRolePolicies, clubRoleActivity, clubNotificationSettings, clubMessagingCredits, clubMarketingCampaigns, clubDailyAuthCodes, clubMessageDeliveries, clubMessageInbound, workerAssociationRequests, clubAdminNotifications, clubOnboardingRecords, promoterOnboardingRecords, clubMedia, messages, privacyConsents, friendRequests, friendships, notifications, roleRequests, clubEmployeeDesignations, patronRanks, approvedShoutOutLibrary, translationSettings, translationOverrides, inboxNotifications, shoutoutAudit, djProfiles, promoterProfiles, shoutoutRecommendations, aiIndex, aiUserNotificationPreferences, aiUserSignals, aiSearchLogs, aiRecommendations, aiMediaEdits, aiDiscoverySources, aiDiscoveryQueue, aiDiscoveryRatingCriteria, aiCrawlRuns, aiCrawlerSchedules, aiDiagnosticsReports, aiAssistantSessions, aiAssistantMessages, patronTemplateVariants, clubTemplateVariants, aiTemplatePromptHistory, minglConnections, minglAudit, minglGists, spotAdCampaigns, chatRooms, chatMessages, mobileTestRuns.

4.4 Target additional collections

Collection	Purpose
organizations	Multi-venue orgs
screens	Physical panels under a venue
auditLogs	Tamper-evident security audit
domainEvents / Pub/Sub	Not stored long-term in Firestore
deviceTelemetry	Time-series → BigQuery primarily; Firestore latest-only
adImpressions	Hot buffer then BigQuery
shippingShipments	Labels, carriers, tracking
entitlements	VIP / subscriptions
apiClients	Developer keys metadata

4.5 Critical document shapes (selected)

users/{uid}

email, displayName, floqrHandle, roles[], clubAdminLocationIds[], country, consents, profileCompleted, Stripe binding refs, schemaVersion.

clubLocations/{locationId}

Identity, geo, adminUids[], adminEmails[], displayScreenFormatIds[], primaryDisplayScreenFormatId, public profile fields, feature flags, templates[], restrictTemplatesToLocationSet.

shoutouts/{id}

clubLocationId, template, texts, media, status, ownerUid, pricing, screen format, approval metadata.

liveContent/{locationId}

Singleton board state: main/sub/template/media/status/approvedAt/duration.

commerceProducts/{id}

Seller, priceCents, inventory, active, productType, requiresShipping, media, license fields.

serviceOrders/{id}

orderType, amounts, paymentModel, shares, shippingDetails, shippingStatus, Stripe ids.

clubAdminAssignments/{locationId_uid}

status: active|revoked, assignedByUid, assignedAt, elected roles snapshot.

4.6 Relationships

- Venue 1—N Screens, Admins, Events, ShoutOuts, Devices.
- User 1—N Orders, ShoutOuts; N—N Mingl connections.
- Order N—1 Product; N—1 Buyer; N—1 Seller.
- liveContent 1—1 Venue (current design) — future: 1—N per screen.

4.7 Indexes

Composite indexes required for:

- shoutouts: clubLocationId + status + createdAt
- serviceOrders: ownerUid + createdAt; sellerUid + createdAt
- commerceProducts: active + category
- minglConnections: participants array-contains + status
- Chat: roomId + createdAt

Export firestore.indexes.json as source of truth in CI.

4.8 Security Rules principles

1. Default deny.
2. Master via custom claim or hard allowlist (migrate fully to claims).
3. Club manager via isClubManager(locationId) (already).
4. Public read only for explicitly public docs.
5. Money collections: client create denied; webhook/admin only.
6. Validate field allowlists on update (prevent privilege escalation).

4.9 Retention / Archival

Data	Hot	Warm	Cold
liveContent	realtime	—	—
shoutouts	90d	GCS JSON 2y	delete/anonymize
chatMessages	30–180d per policy	—	export on request
paymentLedger	7y	—	legal hold
deviceTelemetry	7d Firestore	BigQuery 13m	aggregate
appLogs	30d	—	—

4.10 Search indexing / Caching

- Dual-write to search index on entity change events.
- CDN cache public catalog and club profiles (Cache-Control, surrogate keys).
- Memorystore for hot ad decisioning keys.

4.11 Migration strategy

1. Expand-contract migrations with `schemaVersion`.
 2. Backfill Functions job with checkpointing.
 3. Never break display URL contract (`display.html?location=`).
 4. Alias map (`clubLocationAliases`) for merges — keep.
-

SECTION 5 — Backend

5.1 Firebase platform (current + target)

Product	Current use	Target
Authentication	All patron/admin auth	+ SAML, WebAuthn step-up
Firestore	Primary DB	Primary OLTP; BigQuery analytics
Storage	Media	+ lifecycle + malware scanning
Functions	Business logic	Thin; heavy work → Cloud Run
Hosting	Not primary (Pages used)	Consider Firebase Hosting or Cloud CDN for prod

5.2 Cloud Run

Recommendation: move long-running / CPU (transcode, AI batch, report PDF, device gateway) to Cloud Run services with min instances for display gateway.

5.3 Cloud Scheduler / Tasks / Pub/Sub

- Scheduler: expire shoutouts, rotate daily codes, crawlers (exists).
- Tasks: delayed approve reminders, payout batches.
- Pub/Sub: domain events spine.

5.4 Redis (Memorystore)

Session secondary cache, rate limits, ad bid cache, presence TTLs.

5.5 BigQuery

Canonical analytics warehouse; ELT from Firestore exports + event stream.

5.6 Vertex AI / Gemini

See Section 8. Functions already call Gemini for media/suggest/rank.

5.7 Logging / Monitoring / Secret Manager / Artifact Registry / CI/CD

- Secrets: STRIPE_*, SENDGRID_*, Twilio, Maps — Secret Manager only.

- Artifact Registry for container images.
- CI: GitHub Actions (Functions tests on `main`); expand to full pipelines (Section 13).

Backend recommendation summary

Keep Firebase for auth + OLTP speed; introduce **Cloud Run + Pub/Sub + BigQuery + Memorystore** as the scale backbone without abandoning Firestore prematurely.

SECTION 6 — Mobile Architecture

6.1 Recommendation: Flutter as the primary mobile client

Recommendation: Flutter (iOS/Android) as the long-term mobile standard; share domain logic packages with Flutter Web for selected surfaces.

Why

- One UI codebase for iOS/Android with strong performance for media-heavy nightlife UX.
- Existing web demo can remain for admin/LED HTML player while patrons migrate to Flutter.
- Plugin ecosystem for maps, payments sheets, FCM, camera.

Alternatives

Stack	Pros	Cons
React Native	Web talent reuse	Media/perf variance
Native Kotlin/Swift	Max platform fidelity	2× engineering cost
PWA only	Fast ship	Weak background work, store discovery, device APIs

Migration path

1. Flutter app consumes the same Firebase project + HTTPS APIs.
2. Feature parity order: auth → search → ShoutOut → Mingl → BartR browse/buy → RydR request.
3. Admin remains web-first initially.
4. Deprecate brittle mobile-web-only patron flows once Flutter GA.

6.2 Layering, state, routing, DI

```
Presentation (Widgets)
→ State (Riverpod recommended; Bloc acceptable)
→ Domain UseCases
→ Data Repositories
→ Firebase / HTTP / WebSocket
```

- **DI:** Riverpod overrides in tests; or `get_it` + injectable. No service locators inside widgets.
- **Routing:** `go_router` with `flutter_deep_links` for deep links and universal/app links.
- **State:** immutable models; async providers for repositories; no global singletons of Firestore streams without disposal.

6.3 Offline synchronization

Feature	Offline behavior
Profile	Cached read; queue non-critical edits
BartR browse	Cache last catalog pages
Checkout	Online required
Chat	Local outbox; flush on reconnect
Guest-list / ticket QR	Cache credential for door

Conflict policy: server authoritative for money, approvals, inventory; last-write-wins only for UI prefs.

6.4 Caching / background / performance

- Image disk cache with explicit size caps.
- Isar/Hive for durable small documents.
- FCM background handlers; location only with foreground disclosure for RydR.
- Budgets: cold start <2.5s mid-tier; avoid jank on hero media.

6.5 Accessibility / localization / deep links / notifications

- WCAG-minded contrast (especially BartR light theme); Dynamic Type; semantics.
 - ARB localization; RTL readiness; align with server `translationSettings`.
 - Deep links: venue, product, chat thread, shoutout receipt, payment return.
 - Notification preference center (extend `aiUserNotificationPreferences`).
-

SECTION 7 — Web Architecture

7.1 Current (v29.x)

- Multi-page static HTML/JS on **GitHub Pages** (jadzadco.github.io/shoutout-demo).
- Firebase compat SDKs for Auth/Firestore/Storage/Functions.
- Cache bust via ?v=<package>.
- Surfaces: `index.html`, `patron-portal.html`, `admin.html`, `master-admin.html`, `display.html`, `commerce.html`, `rydr.html`, `pickup.html`, Mingl/guest-list satellites.

7.2 Target strategy

Recommendation — hybrid web:

1. Keep `display.html` as an ultra-stable light player (minimal framework) for Xibo / Amazon Signage Webpage widgets.
2. Rebuild patron + admin portals as either **Flutter Web** (if mobile is Flutter) or **Next.js** (if SEO/hiring optimize for React).
3. Separate **marketing SSR site** (SEO) from **authenticated SPA**.

Concern	Decision
SSR	Marketing + public club/product pages only
PWA	Patron installable shell; never Master Admin
Caching	Hashed assets on CDN; SW excludes checkout/admin
Responsive	Mobile-first patron; dense admin tables OK

Portal map (target)

Patron App · Venue Command Center · Master/Ops · Advertising · Analytics · Developer.

Migration from Pages

Firebase Hosting or Cloud CDN origin → App Check on sensitive calls → preserve display URL contracts and QR deep links forever.

SECTION 8 — AI Architecture

8.1 Principles

- AI accelerates; it does **not** solely authorize money movement, bans, or legal identity.
- Human-in-the-loop for ShoutOut approval and paid ad creatives (unless venue explicitly opts into auto-approve policies).
- Every inference: quota, safety filter, audit log, kill switch.

8.2 Gemini / Vertex AI

Current: Cloud Functions for media enhance, ShoutOut suggest, grammar, location rank, template background gen/modify, admin queue summarize, recommendations, notification preference processing, discovery crawl.

Target: Vertex AI control plane; Gemini multimodal; Vector Search embeddings for semantic retrieval; Feature Store for online ranking features.

8.3 Recommendation / NLU / generation / moderation

- **Recommendations:** hybrid rules + model; features from venue, music, graph, spend, time, geo.
- **FloqAi NLU:** intent → whitelisted tools (Mingl, RydR, BartR, ShoutOut, clubs); no arbitrary code execution.
- **Image/video gen:** template/campaign creatives; prompt history retained (aiTemplatePromptHistory).
- **Moderation:** toxicity/NSFW pre-check + human queue.
- **Semantic search / knowledge graph:** embeddings + future property graph (Patron—Interest—Venue—Event—Artist).

8.4 Predictive / ops / marketing intelligence / agents

- Predict end-of-night ride demand; ShoutOut volume; campaign budget pacing.
 - Agents start **supervised** (draft copy, draft CSR replies). Autonomy only behind flags.
 - Never auto-capture payouts or auto-ban without dual control.
-

SECTION 9 — Hardware

9.1 Portable LED estate

Panel classes already modeled (1ed-*, p125-*). Player: kiosk Chromium or embedded WebView executing the FLOQR display runtime.

9.2 Power, dock, OTA, diagnostics

- Hot-swap packs with SOC/thermal telemetry.
- Dock: power + optional data + mechanical lock.
- Signed OTA with A/B partitions; cohort rollouts.
- Pairing QR → `displayDevices` claim.
- Heartbeat watchdog; auto-reload; RMA and cryptographic retire on decommission.
- Factory provision device identity into secure storage when available.

9.3 Lifecycle

Manufacture → provision → ship → pair → monitor → OTA → RMA → wipe/retire. Inventory SKUs integrate with ERP at Phase 2+.

SECTION 10 — APIs & Integrations

10.1 Standards

Versioned REST/gRPC; idempotency keys; signed webhooks; Secret Manager; OpenAPI 3.1 as partner contract source of truth.

10.2 Integration matrix

Integration	Purpose	Current	Target
Stripe	Checkout, Connect, webhooks	MoR floqr-platform + Connect Accounts v2	Radar, Tax, automated payouts
Google Maps	Places, distance	Discovery usage	Routes/ETA for RydR
Apple Maps	iOS maps	—	Optional
Twilio	SMS/WhatsApp/MFA	Yes	Messaging Services + compliance
SendGrid	Email	Preview/test mailers	Templated transactional
Firebase	Auth/DB/Storage/FCM	Yes	Continues
Xibo	Signage CMS	Webpage → display.html	Keep HTML player
Amazon Signage	DOOH host	Planned	HTML5/package export
Spotify/Apple Music/YouTube	DJ links	URL fields	OAuth verify ownership
Ticketmaster/Eventbrite	Events	Crawl/AI	Official APIs
Meta/TikTok/Instagram	Social/ads	Handles	Marketing APIs later
Ride-share/Robotaxi	Dispatch	Tesla simulation	Adapter interface
Ad exchanges	Programmatic DOOH	—	OpenRTB-like
Weather/Traffic	Ops/RydR	—	Edge-cached
Calendar	Personalization	—	Google/MS OAuth
Shippo/EasyPost/UPS	BartR shipping	—	Labels + tracking webhooks

10.3 Partner API scopes (target)

venue.read, shoutout.write, catalog.read, orders.read, devices.read — OAuth2 client credentials + delegated user tokens.

SECTION 11 — Security

11.1 AuthN / AuthZ / RBAC

- Firebase Auth + **custom claims** for masterAdmin, roles[], venueScopes[] (migrate off email allowlists as sole control).
- Enforce Section 3 matrix server-side; Club Admin requires election assignment + MFA.
- Step-up MFA (SOS2FA generalization) for payouts, offboarding, entity destroy.

11.2 Encryption / keys / PII

- TLS 1.2+; Google-managed encryption at rest; CMEK for enterprise later.
- Minimize PII in analytics; hash phone lookups; public projections stripped.

11.3 GDPR / CCPA / SOC2 / PCI

- Consents (privacyConsents); export/delete runbooks; residency options.
- SOC2 controls: access, change management, logging, vendors.
- PCI **SAQ-A** via Stripe Checkout — never store PAN/CVV.

11.4 Audit / IR / threat model / testing

- Append-only auditLogs for privileged mutations.
- STRIDE per boundary (display, checkout, chat, admin, device).
- Annual external pen test; continuous ASVS-inspired checklist.
- App Check, gateway rate limits, bot detection, Stripe Radar, optional IDV for vendors.

11.5 DR / BC

- RPO ≤ 15 min / RTO ≤ 1 h regional API failover (target).
 - Display offline cache preserves venue continuity.
 - Scheduled Firestore exports; quarterly restore drills.
-

SECTION 12 — Observability

- **Logs:** structured JSON with `correlationId`, `uid`, `locationId`, `orderId`, `deviceId`.
 - **Metrics:** checkout success, approve latency, display heartbeat age, function errors, p95 latency, SMS cost.
 - **Traces:** OpenTelemetry on money and approve paths.
 - **Health:** `/healthz` on Cloud Run; fleet heartbeat dashboard.
 - **Alerting:** sev1 for checkout down / mass device offline.
 - **Cost:** Firestore reads, Functions GB-s, AI tokens, Twilio.
-

SECTION 13 — DevOps

13.1 Git / CI / CD

- Trunk-based; `main` = production; `workspace/*` for long experiments (current).
- Required CI on `main`: Functions tests, rules tests, lint.
- Deploy Functions via controlled workflow + Secret Manager token.
- Web deploy on green `main`; mobile via TestFlight/Play internal.

13.2 Testing / flags / releases

- Unit, emulator integration, E2E critical paths, k6 load on staging.
- Server feature flags (existing gates) + percentage rollout.
- Cloud Run revision traffic split; device OTA cohorts; CDN canary.

13.3 IaC / secrets / rollback

- Terraform for GCP network/alerts/Run services.
 - No secrets in git.
 - Rollback Hosting/Functions revisions; **never break** `display.html?location=` contract; keep package rollback docs.
-

SECTION 14 — Scalability

Stage	Venues	Patrons MAU	Moves
10	Pilot	1k	Current stack
100	City	50k	CDN, indexes, App Check, less client fanout
1,000	Multi-city	500k	Cloud Run extract, Pub/Sub, Memorystore, BigQuery
10,000	National	5M	Multi-region, search service, chat service, device gateway
100,000	Global	50M+	Sharding, regional PII, ads edge, media pipeline

Early hotspots: liveContent fan-in (move to per-screen docs), chat write fanout, catalog scans, sync AI on request path, Pages cache control limits.

Latency/cost: denormalize reads; cache catalogs; lifecycle ephemeral media; prefer push over SMS.

SECTION 15 — Technical Roadmap

MVP (now → ~90 days)

- Club Admin RBAC + back-nav containment (in progress/shipped pieces).
- Public BartR browse; auth at purchase; white/black/red + signed-in dark theme.
- Display reliability (template className fixes, idle/brand loops).
- CI gate before preview emails.
- Ledger correctness + test payment purge tools.

Phase 1 — Production hardening

Claims RBAC · App Check · Device gateway + offline player · BigQuery · OpenAPI · Flutter beta.

Phase 2 — Scale products

Extract chat/search · FloqMedia campaign+impressions · BartR shipping integrations · Real RydR adapters · US-EU multi-region.

Phase 3 — Platform

Programmatic DOOH · Template marketplace · Org/Regional admin · Hardware OTA at volume · Supervised AI agents.

Future

Smart-city presence · Hospitality PMS · Robotaxi-native logistics · On-device AI assist.

Technical debt (named)

Static multi-page JS without package boundaries; broad client Firestore writes; email allowlists; Pages vs managed CDN; incomplete BartR public UX; RydR simulation-only; collection sprawl.

Refactor sequence

Domain packages → write facades → event bus → extract by SLO pain → replace demo portals incrementally.

Top risks

Risk	Mitigation
Display downtime	Offline cache, dual network
Stripe misconfig	Idempotent webhooks, recon

UGC abuse	AI + human moderation
AI/SMS cost	Quotas, prefs
Single-region outage	Multi-region plan
Client key misuse	App Check, rotate, backend facades

SECTION 16 — Engineering Recommendations

16.1 Weaknesses

1. Browser-direct Firestore as de facto public API.
2. Domain boundaries not mechanically enforced.
3. Chat/search/devices not yet independently scalable.
4. Observability below institutional bar.
5. Hardware not fleets-grade in software.

16.2 Bottlenecks

Listen fanout to displays; chat writes; product collection scans; synchronous Gemini on UX path; GitHub Pages operational limits.

16.3 Architectural risks

Money+social+DOOH cognitive coupling; Master vs Club privilege confusion (partially remediated); embed URL contract breakage.

16.4 Priority improvements

1. Domain packages + server write facades
2. Display offline + device gateway
3. Claims-based RBAC
4. Public BartR + shipping state machine
5. BigQuery analytics
6. Flutter patron app
7. Pub/Sub domain events
8. Extract chat/search when metrics demand

16.5 Alternatives to revisit later

Postgres/Supabase OLTP if relational reporting dominates; full AWS only for mandate customers; MQTT device mesh vs HTTPS gateway at $\geq 10k$ devices.

16.6 Decade watchlist

On-device multimodal moderation · robotaxi API standardization · programmatic DOOH protocols · passkeys/continuous auth · confidential computing · spatial venue UX · energy-aware LED scheduling · verifiable age/VIP credentials · compliance-first new payment rails.

Appendix A — Current Cloud Functions (inventory)

AI / discovery / auth / admin: scheduledAiDiscoveryCrawl, runFloqrDiscoveryCrawl, requestEmailOtp, verifyEmailOtp, assignClubAdmin, aiSearch, aiExtractPublicSourceUrl, aiEnhanceShoutOutMedia, aiSuggestShoutOut, aiSuggestGrammarCorrection, aiRankLocations, aiGenerateTemplateBackground, aiModifyTemplateBackground, aiSummarizeAdminQueue, aiGenerateRecommendation, aiProcessNotificationPreferences, plus preview/test email callables.

Commerce / payments / mobility / ads: getFloqrConnectStatus, createFloqrConnectOnboardingLink, createFloqrCheckoutSession, getFloqrClubCheckoutReadiness, cancelFloqrCheckoutOrder, clearUnpaidFloqrCheckouts, purgeFloqrTestPayments, stripeFloqrWebhook, expireLiveShoutouts, publishFloqrFollowerCampaign, requestTeslaRobotaxiPickup, catalog/ad seed helpers.

Messaging / marketing / gates: getClubDailyAuthCode, sendClubTestMessage, onShoutoutCreatedNotifyClub, rotateClubDailyAuthCodes, messagingInboundWebhook, messaging credit/campaign functions, setPatronFeatureGates, setEntityAppEnabled, setVenueFeatureGates, offboardEntity.

Appendix B — Governance

- Canonical path: docs/FLOQR-TECHNICAL-BIBLE.md
- Review cadence: quarterly or on Tier-1 change
- Deviations require ADR in docs/adr/NNNN-title.md
- Owners: acting CTO / Chief Architect

Appendix C — Related docs in repo

README.md, STANDARD-AFTER-DEPLOYMENT.md, STRIPE-INTEGRATION-PLAN.md, STRIPE-DEPLOYMENT.md, XIBO-ZEBBIES-FOOTBALL-SETUP.md, TESLA-ROBOTAXI-SIMULATION-API.md, DJ-BARTR-MIXCLOUD.md, TEXT-LIMITS-V29-08-4.md, DEPLOYMENT-V29-*, ROLLBACK-V29-*.

End of FLOQR Technical Bible v1.0.0